

AI and ML Foundations in Six Weeks

Learn ML as a decision-making pipeline rather than a catalogue of model names. Define the prediction time, label, objective, data split, baseline, and error costs before optimizing a model. Reliable evaluation and data quality usually matter more than an impressive architecture diagram.

For: Software engineers and students who want a grounded ML foundation for interviews and projects, with emphasis on problem framing, valid evaluation, and simple baselines.

Suggested pace: 6 weeks with Python and basic algebra | Introductory machine learning

What you will be able to do

- Translate a product need into a prediction task and measurable decision.
- Explain regression, classification, loss, optimization, and regularization.
- Choose evaluation metrics and splits that reflect deployment.
- Build a small reproducible baseline and diagnose its errors honestly.

Build the right mental model

The target is a contract

Specify one prediction unit, the moment it is made, the future outcome used as a label, and what action follows. Features must be available at prediction time. A convenient proxy such as clicks may not fully represent the business objective.

Start simple and compare

A heuristic or simple linear/tree model gives a baseline for value and complexity. Better training fit alone is not progress. Inspect validation performance, uncertainty, and important cohorts before choosing a more complex model.

Evaluation is part of the model

Random, grouped, and temporal splits test different generalization claims. Fit preprocessing inside the training boundary. Keep a final test set isolated from repeated tuning, and use metrics tied to the consequences of errors.

Study with evidence.

Use one modest public or synthetic dataset and maintain a reproducible experiment log. For every model, record the task, features, split, baseline, metrics, and failure cases. Avoid changing all of these at once and attributing the result solely to the model.

Weeks 1-2: Problem and mathematical basics

Review probability, vectors, basic derivatives, regression, logistic classification, and gradient descent. Frame one prediction problem and write its label and feature-time contract. Establish a simple baseline.

Deliverable and check: A small numeric example and a baseline evaluation that can be reproduced from a clean environment.

Week 3: Data and generalization

Study leakage, missing values, categorical features, regularization, and learning curves. Compare training and validation behavior. Use a pipeline so fitted transformations respect the split.

Deliverable and check: A leakage checklist and an explanation of underfitting, overfitting, and optimization failure in the experiment.

Weeks 4-5: Models and metrics

Compare a linear model and a tree-based baseline under the same split. Study precision, recall, ranking, calibration, and error costs. Analyze slices and choose thresholds using validation data only.

Deliverable and check: A model comparison with consistent inputs and a justified operating point, not just the highest accuracy.

Week 6: Deployment reasoning

Explain feature availability, model versioning, latency, fallbacks, monitoring, and delayed labels. Run a mock case discussion and identify what the small project does not yet prove about production behavior.

Deliverable and check: An interview-ready project narrative with results, limitations, and a safe rollout proposal.

A repeatable practice session

Spend about 15 minutes recalling the previous topic without notes, 45 minutes on today's problem or lab, and 15 minutes explaining and checking the result. Record the failed assumption, not just the final answer. Revisit weak material after a few days and again the following week.

Apply it. Explain it.

Design a churn prediction baseline

A subscription product wants to identify customers likely to leave in the next thirty days. Historical records contain activity before and after the date on which a prediction would have been made.

A worked approach

1. Choose a prediction date and a future thirty-day label window. Define churn precisely and include only features available by the prediction date. Exclude cancellation outcomes and future activity from the feature set.
2. Use a chronological evaluation split and compare with a simple recency or activity baseline. Handle repeated customers and feature snapshots deliberately so the experiment tests the intended future-use scenario.
3. Evaluate ranking and a capacity-limited intervention threshold, considering false alarms and intervention cost. Explain that correlation does not prove contacting a customer prevents churn; measuring intervention effect requires additional experimental design.

Expected result

The project has a defensible prediction boundary and baseline. It avoids an unrealistically easy evaluation caused by future information and separates predictive quality from the causal value of an intervention.

Verify the result

- Could every feature exist at the real prediction moment?
- Are evaluation labels mature and consistently defined?
- Does the metric support the action the product will take?

Extend the exercise

A marketing policy changes after deployment. Discuss whether changed outcomes reflect model drift, changed customer behavior, or the intervention itself, and what evidence would distinguish them.

Make the learning stick.

When should I avoid ML?

When a simpler rule already meets the objective, reliable labels are unavailable, maintenance cost outweighs likely value, or the decision cannot be evaluated responsibly. A baseline helps reveal whether ML adds enough benefit.

Does lower training loss mean a better model?

Not necessarily. The model may overfit, exploit leakage, or optimize a poor proxy. Compare valid evaluation performance, operational constraints, and error consequences before declaring improvement.

Why is the test set not for repeated tuning?

Repeated choices based on test results leak information into development and overstate generalization. Use training and validation for decisions, then reserve an appropriately isolated test evaluation for the final assessment.

Avoid these traps

- Choosing a model before defining the prediction and action.
- Including future information in features or preprocessing.
- Reporting one aggregate metric without a baseline or error analysis.

Ready for the next step?

- Prediction time, labels, and available features are explicit.
- A simple baseline is evaluated fairly.
- Metrics and thresholds match error costs and constraints.
- The project narrative includes limitations and deployment risks.

Reliable next reads

- [Google Machine Learning Crash Course](#)
- [scikit-learn model evaluation](#)
- [Rules of Machine Learning](#)

An independent preparation guide, not an employer-endorsed syllabus or a guarantee of an offer. Company examples are practice contexts, not verified past interview questions. Use only public, synthetic, or authorized material in exercises.